

Python 异步编程完全指南Python 异步编程完全指南

文档类型: 技术教程 适用版本: Python 3.11+ 难度等级: 中级
最后更新: 2026-06-15**文档类型**: 技术教程
适用版本: Python 3.11+
难度等级: 中级
最后更新: 2026-06-15
文档类型: 技术教程 适用版本: Python 3.11+ 难度等级: 中级
最后更新: 2026-06-15**文档类型**: 技术教程
适用版本: Python 3.11+
难度等级: 中级
最后更新: 2026-06-15

1. 异步编程基础1. 异步编程基础

1.1 为什么需要异步? 1.1 为什么需要异步?

在传统的同步编程模型中, 当程序执行 I/O 操作 (网络请求、文件读写、数据库查询) 时, 线程会被阻塞, 等待操作完成。对于 I/O 密集型应用 (如 Web 服务器), 这会导致严重的资源浪费。在传统的同步编程模型中, 当程序执行 I/O 操作 (网络请求、文件读写、数据库查询) 时, 线程会被阻塞, 等待操作完成。对于 I/O 密集型应用 (如 Web 服务器), 这会导致严重的资源浪费。

1.2 并发模型对比1.2 并发模型对比

模型模型	代表技术代表技术	优点优点	缺点缺点	适用场景适用场景
多线程多线程	threading`threading`	利用多核, 编程模型简单	GIL 限制, 上下文切换开销	CPU 密集型CPU 密集型
多进程多进程	multiprocessing`multip	绕过 GIL, 真正并行	内存开销大, IPC 复杂	内存密集型CPU 密集型
异步 I/O异步 I/O	asyncio`asyncio`	低开销, 高并发	需要 async/await 语法	I/O 密集型I/O 密集型
回调回调	Twisted / Tornado	早期高性能方案	回调地狱, 难以调试	遗留系统遗留系统

2. async/await 语法2. async/await 语法

2.1 核心关键字2.1 核心关键字

```
import asyncio

async def fetch_data(url: str) -> dict:
    """__ async def """
    print(f"{url}")
    # await
    await asyncio.sleep(1) #
    print(f"{url}")
    return {"status": 200, "data": f" {url} "}

async def main():
    #
    tasks = [
        fetch_data("https://api.example.com/users"),
        fetch_data("https://api.example.com/posts"),
        fetch_data("https://api.example.com/comments"),
    ]
    results = await asyncio.gather(*tasks)
    for result in results:
        print(result)

#
asyncio.run(main())
```

2.2 可等待对象 (Awaitables) 类型2.2 可等待对象 (Awaitables) 类型

类型类型	说明说明	创建方式创建方式	可多次 await? 可多次 await?
Coroutine**Coroutine**	async def 函数返回值`async d	async def func()`async def fui	只能一次 只能一次
Task**Task**	被调度执行的协程被调度执行的	asyncio.create_task()`asyncio.	
Future**Future**	低层级，代表将来完成的操作低	loop.create_future()`loop.creæ	

3. 并发控制模式3. 并发控制模式

3.1 信号量 (Semaphore) 限制并发数3.1 信号量 (Semaphore) 限制并发数

```
import asyncio

async def bounded_fetch(
    sem: asyncio.Semaphore,
    url: str,
) -> dict:
    """
    async with sem:
        print(f" {url} ...")
        await asyncio.sleep(0.5)
        return {"url": url, "result": "ok"}

async def main():
    # 5
    semaphore = asyncio.Semaphore(5)
    urls = [f"https://api.example.com/item/{i}" for i in range(20)]

    tasks = [bounded_fetch(semaphore, url) for url in urls]
    results = await asyncio.gather(*tasks)
    print(f" {len(results)} ")
```

3.2 生产者 - 消费者模式3.2 生产者 - 消费者模式

```

import asyncio
from collections.abc import AsyncIterator

async def producer(queue: asyncio.Queue) -> None:
    """
    for i in range(10):
        await asyncio.sleep(0.1)
        item = f"-{i:03d}"
        await queue.put(item)
        print(f" → {item}")

    #
    for _ in range(3):
        await queue.put(None)

async def consumer(queue: asyncio.Queue, name: str) -> None:
    """
    while True:
        item = await queue.get()
        if item is None:
            queue.task_done()
            break
        await asyncio.sleep(0.3) #
        print(f" [{name}] ← {item}")
        queue.task_done()

async def main():
    queue: asyncio.Queue[str | None] = asyncio.Queue(maxsize=5)

    # 1 + 3
    await asyncio.gather(
        producer(queue),
        consumer(queue, "A"),
        consumer(queue, "B"),
        consumer(queue, "C"),
    )

```

4. 异步上下文管理器4. 异步上下文管理器

```

import asyncio
from typing import AsyncIterator

class AsyncDatabase:
    """

    async def connect(self) -> None:
        await asyncio.sleep(0.1)
        print("")

    async def disconnect(self) -> None:
        await asyncio.sleep(0.1)
        print("")

    async def execute(self, sql: str) -> list[dict]:
        await asyncio.sleep(0.05)
        return [{"sql": sql, "rows": 42}]

class AsyncDBConnection:
    """

    def __init__(self, dsn: str) -> None:
        self._dsn = dsn
        self._db = AsyncDatabase()

    async def __aenter__(self) -> AsyncDatabase:
        await self._db.connect()
        return self._db

    async def __aexit__(self, *args) -> None:
        await self._db.disconnect()

async def main():
    async with AsyncDBConnection("postgresql://localhost/test") as db:
        result = await db.execute("SELECT count(*) FROM users")
        print(result)
#

```

5. 性能基准测试5. 性能基准测试

5.1 HTTP 客户端对比5.1 HTTP 客户端对比

方案	1000 请求耗时	内存占用	代码复杂度
requests (同步)	65.3s	12MB	低
requests + ThreadPool	8.2s	45MB	中
aiohttp (asyncio)	2.1s	18MB	中
httpx (async)	2.3s	22MB	低

测试条件：1000 次 HTTP GET 请求，目标延迟 100ms

5.2 FastAPI vs Flask 对比

指标	Flask + Gunicorn	FastAPI + Uvicorn
每秒请求数 (RPS)	1,245	8,930
P99 延迟	480ms	62ms
内存占用	120MB	85MB
启动时间	2.1s	1.4s

6. 常见陷阱与最佳实践

6.1 避免阻塞事件循环

```
#
async def bad_handler():
    import time
    time.sleep(5) #
    return "done"

#
async def good_handler():
    loop = asyncio.get_running_loop()
    await loop.run_in_executor(None, time.sleep, 5)
    return "done"

#
async def best_handler():
    await asyncio.sleep(5)
    return "done"
```

6.2 任务管理检查清单

检查项检查项	说明说明
确保 Task 被 await 或存储引用确保 Task 被 awai	防止 Task 被 GC 导致静默失败防止 Task 被 GC 导致静
使用 asyncio.timeout()使用 `asyncio.tim	防止协程无限挂起防止协程无限挂起
处理 CancelledError处理 `CancelledError	优雅关闭资源优雅关闭资源
避免在 __init__ 中使用 async避免在 `__init__	使用工厂方法或 __init_subclass__使用工厂方法或 `__
设置 PYTHONASYNCIODEBUG=1设置 `PYTHONAS	开发时检测常见错误开发时检测常见错误

7. 总结7. 总结

Python 异步编程的核心是理解事件循环和协程调度。`asyncio` 为 I/O

密集型应用提供了极高并发的可能。关键点:

1. `async def` 声明协程, `await` 交出控制权**`async def`** 声明协程, **`await`** 交出控制权
2. `asyncio.gather()` 用于并发执行, `Semaphore` 控制并发量**`asyncio.gather()`**
用于并发执行, **`Semaphore`** 控制并发量
3. 永远不要让同步阻塞代码进入事件循环永远不要让同步阻塞代码进入事件循环
4. 异步上下文管理器 (`__aenter__`/`__aexit__`) 是资源管理的标准方式异步上下文管理器 (`__aenter__`/`__aexit__`) 是资源管理的标准方式
5. 选择 `aiohttp` 或 `httpx` 作为异步 HTTP 客户端选择 `aiohttp`` 或 `httpx`` 作为异步 HTTP 客户端

本文档由后端研发组编写, 欢迎提交 PR 改进内容。